An Adaptive-Precision, Dual-Step CORDIC Engine for Unified Sine, Cosine, and Arctangent Computation

Md. Atair Rahman Alvi
Department of Electrical and Electronic Engineering
BRAC University, Dhaka, Bangladesh
atair.rahman@bracu.ac.bd

***Abstract*—This paper presents an adaptive-precision, dual-step CORDIC architecture for unified circular-mode trigonometric computation, covering sine, cosine (rotation mode) and arctangent (vectoring mode) within a single reconfigurable datapath. Rather than fixing the iteration depth to a worst-case bound, the proposed engine terminates the iterative process early once a stage-dependent residual falls below a precision-mode threshold, and further reduces per-cycle cost through a dual-step-per-cycle (DSPC) scheduling scheme that commits two CORDIC micro-rotations per clock edge. The design is described in synthesizable Verilog and verified against a bit-accurate software reference using a held-out set of 5,000 rotation and 5,000 vectoring vectors disjoint from the set used to tune the precision-mode parameters, plus a deterministic boundary/worst-case set that exposed and led us to fix a genuine branch-cut bug in the output-angle correction (a 2π error at exact-axis vectoring inputs) and to characterize a fixed-point dynamic-range floor below which accuracy degrades sharply (operand magnitudes under 2^{-13} in the chosen Q4.20 format). On the held-out set, combining dual-step scheduling with residual-based early termination reduces mean cycle count by 54.3%, 51.2%, and 47.0% relative to a fixed-depth baseline for the High-, Medium-, and Low-precision modes; *however*, OpenSTA static timing analysis on the SKY130-mapped netlists shows that the added combinational depth lowers the maximum clock frequency from 84.2 MHz (baseline) to 33.3 MHz (combined) – proportionally more than the cycle savings – so real throughput ends up 13–25% *lower* than the fixed-depth baseline across the three precision modes, despite the cycle-count reduction. Maximum observed error stays within 7.9×10^{-5}, 3.2×10^{-4}, and 1.8×10^{-3} for the three modes respectively. We report the throughput result plainly because it is, to our knowledge, a rarely shown but practically important caution for CORDIC scheduling research: cycle-count metrics without post-synthesis timing closure can be actively misleading. Residual-based early termination alone, which does not alter the per-iteration combinational depth, is the mechanism that survives this analysis with a modest but real benefit. We additionally report real SKY130-mapped area (17,702–27,230 μm^2 across configurations), static leakage power, iCE40 FPGA-targeted logic-resource utilization (LUT4/carry/flip-flop counts), and a word-length sensitivity sweep.**

***Index Terms*—CORDIC, sine/cosine, arctan2, adaptive precision, early termination, FPGA/ASIC hardware, shift-add arithmetic.**

I. Introduction

Sine, cosine, and arctangent evaluation are fundamental primitives in digital signal processing, communication systems, robotics, motor-control angle tracking, and graphics pipelines, underlying transforms such as the DFT, DCT, and FFT as well as direct digital frequency synthesis [3], [4]. Lookup-table and polynomial-approximation methods trade memory or multiplier cost directly against precision [9], while the shift-add nature of the Coordinate Rotation Digital Computer (CORDIC) algorithm, introduced by Volder [1] and later unified across circular, linear, and hyperbolic coordinate systems by Walther [2], makes it an attractive multiplier-free alternative for resource-constrained hardware. CORDIC's regular, iterative shift-add structure has since been adopted across a wide range of DSP and signal-processing hardware [3], [4], [5], and continues to be revisited for FPGA- and ASIC-oriented sine/cosine generation [9], [5].

The central drawback of a conventional CORDIC engine is that its iteration count is fixed by the worst-case precision target: every input is processed through the full micro-rotation sequence even when the required output accuracy could be reached earlier. This has motivated a substantial body of work on reduced-iteration and scale-free circular-mode variants, including mixed-scaling-rotation CORDIC [14], Taylor-series-based scaling-free rotators [17], [19], hybrid CORDIC-multiplier synthesizer/mixer architectures [18], and angle-recoded low-latency variants for waveform generation [16], [9], as well as generalized hyperbolic-mode CORDIC supporting arbitrary logarithm/exponential bases [15]. Separately, a distinct line of adaptive and reduced-iteration CORDIC variants [10], [11], [12], [13] targets a specific downstream application (recurrent-network activation functions [11], [12], LSTM acceleration [12], or signal-processing front ends [8]) and typically addresses either reduced iteration count or improved per-cycle throughput, but not both simultaneously within a single reconfigurable, precision-selectable engine. Most of the scale-free and angle-recoded variants above reduce iteration count *algorithmically*, at the cost of additional per-iteration Taylor or recoding logic, and to our knowledge none combine a residual-driven adaptive-precision criterion with a scheduling-level (rather than algorithmic) cycle-cost reduction of the kind proposed here. Separately, root- and power-extraction CORDIC variants [6], [7] have explored adaptive-precision and higher-radix scheduling for hyperbolic-mode functions, but circular-mode trigonometric functions – arguably CORDIC's most common use case [4], [5] – have

not received the same combined treatment.

This paper targets circular-mode CORDIC specifically – sine, cosine, and arctangent – and proposes a design that combines two complementary acceleration mechanisms:

1) A *residual-based early-termination* mechanism, in which the iteration process for each mode (rotation or vectoring) stops once its natural convergence residual (z_i for rotation, y_i for vectoring) falls below a precision-mode-dependent threshold, subject to a minimum iteration floor.
2) A *dual-step-per-cycle (DSPC)* scheduling scheme that commits two consecutive CORDIC micro-rotations to the state registers within a single clock edge, halving the cycle cost of the regular iteration loop whenever the stage state permits.

Because circular-mode CORDIC does not require the repeated-index correction that hyperbolic-mode CORDIC needs for convergence [7], the dual-step scheduling here is structurally simpler than in prior repeated-index hyperbolic designs, and applies uniformly across the full iteration sequence. Full angular range is recovered with a standard quadrant-folding pre/post-processing stage, extending the native $[-\pi/2, \pi/2]$ rotation-mode convergence range and the native first/fourth-quadrant vectoring-mode range to the complete $[-\pi, \pi)$ domain.

The contributions of this work are:

1) An adaptive-precision, residual-driven early-termination criterion specialized to circular-mode rotation and vectoring CORDIC, with three selectable precision modes (High/Medium/Low) whose target depths, floors, and thresholds were tuned via a fixed-point sweep against a floating-point software reference.
2) A dual-step-per-cycle (DSPC) scheduling scheme for the circular-mode micro-rotation loop, requiring no repeated-index handling.
3) A synthesizable Verilog implementation verified against a bit-accurate software model over a held-out set of 5,000 rotation and 5,000 vectoring vectors (disjoint from the parameter-tuning set) plus a deterministic boundary/worst-case set, with a full ablation study isolating the individual and combined contributions of early termination and dual-step scheduling to cycle count, real post-synthesis critical-path timing, and synthesized area across SKY130 ASIC and iCE40 FPGA targets.

II. Mathematical Formulation

A. Circular CORDIC Rotation and Vectoring Modes

The circular-mode CORDIC iteration is

$$x_{i+1} = x_i - \sigma_i y_i 2^{-i} \tag{1}$$

$$y_{i+1} = y_i + \sigma_i x_i 2^{-i} \tag{2}$$

$$z_{i+1} = z_i - \sigma_i \tan^{-1}(2^{-i}) \tag{3}$$

where, in rotation mode, $\sigma_i = \mathrm{sign}(z_i)$ drives the angular residual z_i toward zero, and in vectoring mode, $\sigma_i = -\mathrm{sign}(y_i)$ drives the vertical residual y_i toward zero. Unlike hyperbolic CORDIC, no iteration index needs to be repeated for convergence, so i simply runs $0, 1, 2, \ldots$ Over n iterations the cumulative gain is

$$A_n = \prod_{i=0}^{n-1} \sqrt{1 + 2^{-2i}}, \qquad K_n = 1/A_n. \tag{4}$$

1) Rotation mode – sine and cosine: With $x_0 = K_n$, $y_0 = 0$, $z_0 = \theta$ (for $\theta \in [-\pi/2, \pi/2]$), the outputs converge to

$$x_n \to \cos\theta, \qquad y_n \to \sin\theta. \tag{5}$$

Pre-scaling x_0 by K_n (rather than post-scaling the result) avoids an explicit multiplier for gain compensation, mirroring the scale-factor handling used in other CORDIC-based function engines [6].

2) Vectoring mode – arctangent: With x_0, y_0 given (for $x_0 \geq 0$), the outputs converge to

$$z_n \to \tan^{-1}(y_0/x_0), \qquad x_n \to K_n \sqrt{x_0^2 + y_0^2}. \tag{6}$$

B. Quadrant Folding for Full-Range Support

Native hyperbolic-free circular CORDIC converges for $\theta \in [-\pi/2, \pi/2]$ (rotation) and $x_0 \geq 0$ (vectoring). Full-range support is recovered with the following standard folding rules, evaluated combinationally before the iterative core and undone on its output:

- Rotation: for $\theta \in (\pi/2, \pi)$, evaluate at $\pi - \theta$ and negate the returned cosine; for $\theta \in (-\pi, -\pi/2)$, evaluate at $\theta + \pi$ and negate both returned cosine and sine.
- Vectoring: for $x_0 < 0$, evaluate at $(-x_0, -y_0)$ and add or subtract π from the resulting angle depending on its sign.

This preprocessing stage is a standard technique (also used for range extension in hyperbolic CORDIC engines [6]) and is not itself a contribution of this work; it is included in the synthesized design and its gate cost is reflected in the results of Section IV.

III. Adaptive Precision Through Residual-Based Early Termination

A. Stage-Dependent Termination Criteria

Let $P \in \{\mathrm{High}, \mathrm{Medium}, \mathrm{Low}\}$ denote the selected precision mode. For each stage $s \in \{\mathrm{Rot}, \mathrm{Vec}\}$, a nominal target depth $i_{\mathrm{tar},s}(P)$ upper-bounds the executed iteration count, and a minimum floor $i_{\mathrm{floor},s}(P)$ prevents premature termination from transient residual fluctuations in fixed-point arithmetic. Once $i \geq i_{\mathrm{floor},s}(P)$, the stage may terminate early once its residual criterion is satisfied:

$$\text{Rotation:} \quad |z_i| < \tau_{\mathrm{Rot}}(P) \tag{7}$$

$$\text{Vectoring:} \quad |y_i| < \tau_{\mathrm{Vec}}(P) \tag{8}$$

Both criteria are evaluated on the candidate next-state value, so termination is only allowed after the current micro-iteration has completed.

TABLE I
REPRESENTATIVE PRECISION-MODE SWEEP (ROTATION STAGE); VECTORING STAGE SWEEP FOLLOWED THE SAME PROCEDURE WITH CLOSELY MATCHING THRESHOLDS.

Depth/Floor/Threshold	Avg. err.	Max. err.	Mean cyc.
18/12/2^{-16} (High)	6.80×10^{-6}	1.84×10^{-5}	15.23
18/12/2^{-14}	2.84×10^{-5}	6.26×10^{-5}	13.66
14/10/2^{-14} (Medium)	3.10×10^{-5}	1.08×10^{-4}	13.21
14/10/2^{-12}	1.13×10^{-4}	2.42×10^{-4}	11.67
11/8/2^{-12} (Low)	2.91×10^{-4}	9.49×10^{-4}	10.61
11/8/2^{-10}	4.27×10^{-4}	9.65×10^{-4}	9.65

TABLE II
SELECTED PRECISION-MODE PARAMETERS (BOTH STAGES SHARE THE SAME NOMINAL DEPTH/FLOOR; PER-STAGE THRESHOLDS ARE WITHIN $2\times$ OF ONE ANOTHER AND ARE SET EQUAL FOR DATAPATH SIMPLICITY).

Mode	Target depth i_{tar}	Floor i_{floor}	Threshold τ
High	18	12	2^{-16}
Medium	14	10	2^{-14}
Low	11	8	2^{-12}

B. Parameter Selection

Target depths, floors, and thresholds were selected via a fixed-point sweep (24-bit operands, Q4.20 format, 18-entry $\tan^{-1}(2^{-i})$ table) against 500 randomly sampled angles (rotation) and 500 randomly sampled (x, y) pairs (vectoring), comparing the fixed-point output to a double-precision software reference. Table I summarizes representative points from this sweep; the selected operating points (bolded) were chosen as the deepest threshold at each nominal depth/floor pair before mean-cycle savings began to plateau.

The resulting parameters used in the hardware implementation are given in Table II.

IV. HARDWARE ARCHITECTURE AND IMPLEMENTATION RESULTS

A. Overall Architecture

Figure 1 shows the top-level architecture. A single reconfigurable engine implements both rotation and vectoring modes, selected by a one-bit `op` signal; a two-bit `prec_sel` input selects the active precision mode. The quadrant-folding stage described in Section II-B sits combinationally in front of the iterative core and is undone on the core's output.

B. Dual-Step-Per-Cycle (DSPC) Datapath

Figure 2 shows the stage-level scheduling. In the DSPC configuration, the core computes a first candidate next state $(x_{i+1}, y_{i+1}, z_{i+1})$, immediately uses it as the input to a second combinational micro-iteration producing $(x_{i+2}, y_{i+2}, z_{i+2})$, and evaluates the residual/floor/target termination logic against *both* candidates: if the stopping condition is already satisfied after the first of the two micro-iterations, only that one is committed (avoiding overshooting the target depth by one), otherwise both are committed in a single clock edge.

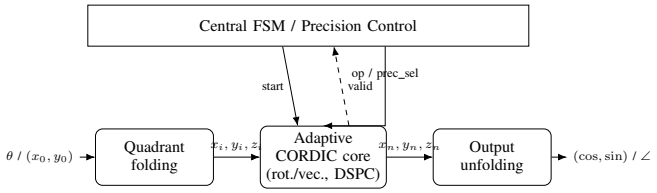


Fig. 1. Top-level hardware architecture: a Central FSM sequences the `start`/`valid` handshake and distributes `op` (mode) and `prec_sel` (precision) to the single reconfigurable adaptive CORDIC core; quadrant folding/unfolding sit combinationally around it (no separate control signal, per Section II-B).

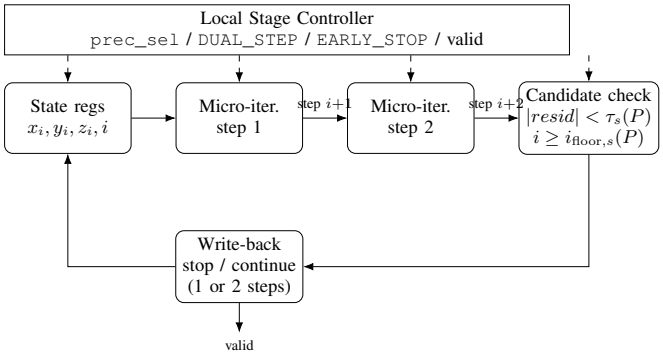


Fig. 2. Stage-level DSPC datapath and control flow: two micro-iterations are evaluated combinationally per cycle; the Local Stage Controller distributes the active precision mode and the `DUAL_STEP`/`EARLY_STOP` configuration, and the write-back stage commits one or both candidate states depending on the termination check.

C. Verification Methodology

Figure 3 summarizes the overall research methodology, from the floating-point reference model through to the tables and figures reported in this paper, and the tool used at each stage.

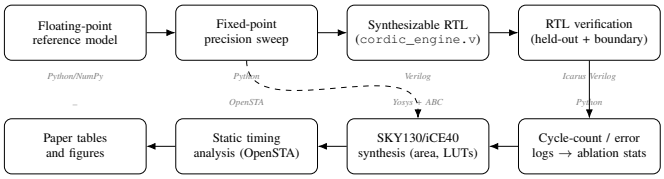


Fig. 3. End-to-end research methodology (left-to-right, then wrapping right-to-left). The dashed arrow indicates that the same precision-mode parameters tuned against the floating-point reference (Stage 2) are what is instantiated in the RTL that gets synthesized (Stage 6) – i.e. the area/cycle results in Section IV are for the exact configurations whose accuracy was characterized in Section III, not a separately re-tuned design.

The design (`cordic_engine.v` and the `cordic_top.v` folding wrapper) was described in synthesizable Verilog and verified with Icarus Verilog against the bit-accurate Q4.20 software reference. Four configurations were built by toggling two compile-time parameters, `DUAL_STEP` and `EARLY_STOP`: *Baseline* (both disabled – fixed-depth, single-step), *Early-stop only*, *Dual-step only*, and *Combined*.

Tuning/evaluation split. The 500-vector set used in Section III to select the target-depth/floor/threshold parameters (seed 1) is used only for that purpose. All results reported below use a disjoint, 10× larger *held-out* set – 5,000 rotation angles and 5,000 vectoring pairs, drawn with an independent seed – so that the reported accuracy and cycle statistics are not measured on the same data used to pick the thresholds.

Boundary/worst-case set. Uniform random sampling, even at N=5,000, has zero probability of hitting exact quadrant boundaries, exact zeros, or axis-aligned inputs. We additionally built a deterministic set (411 angles, 425 (x, y) pairs) covering $\{0, \pm\pi/2, \pm\pi\}$ and their immediate fixed-point neighbors, a dense 400-point angular grid, and (x, y) pairs at the extremes and axes of the tested magnitude range. Running this set through the RTL surfaced two genuine findings that random testing alone did not: (1) a branch-cut bug in the vectoring output-angle correction – for $x_0 < 0$ with y_0 exactly zero, the original correction picked the wrong side of the $\pm\pi$ branch cut based on the CORDIC engine's own noise-sensitive residual sign rather than the sign of the original input, producing an angle error of a full 2π; we fixed this by driving the correction from the registered sign of y_0 itself (Section II-B), which matches the atan2 convention that $y = +0$ maps to $+\pi$ for negative x, and re-ran all four configurations after the fix. (2) A fixed-point dynamic-range floor: for operand magnitudes below roughly 2^{-13} ($\approx 1.2 \times 10^{-4}$) in this Q4.20 format, the vectoring output's absolute error grows from the 10^{-5}–10^{-4} range into the 10^{-2}–10^{-1} range and beyond, because shifted copies of a near-LSB operand underflow to zero within the first few micro-iterations; this is an inherent fixed-point range limitation rather than an algorithmic defect, and we report it as an explicit usable-input-range bound rather than folding it into the random-vector error statistics below. All numbers in Table III and Section IV-E–IV-H use the post-fix RTL.

TABLE III
ABLATION STUDY: MEAN CYCLES AND ERROR, ROTATION (ROT.) AND VECTORING (VEC.) STAGES, FROM RTL SIMULATION OVER A HELD-OUT 5,000+5,000 EVALUATION SET (DISJOINT FROM THE TUNING SET), POST BUG-FIX.

Config.	Mode	Rot. cyc.	Vec. cyc.	Rot. max err.	Vec. max err.
Baseline	High	18.00	18.00	1.49×10^{-5}	2.82×10^{-5}
	Medium	14.00	14.00	1.26×10^{-4}	1.28×10^{-4}
	Low	11.00	11.00	9.77×10^{-4}	9.82×10^{-4}
Early-stop only	High	15.28	16.89	2.16×10^{-5}	7.91×10^{-5}
	Medium	13.18	13.72	1.26×10^{-4}	3.23×10^{-4}
	Low	10.66	10.88	9.46×10^{-4}	1.78×10^{-3}
Dual-step only	High	9.00	9.00	1.49×10^{-5}	2.82×10^{-5}
	Medium	7.00	7.00	1.26×10^{-4}	1.28×10^{-4}
	Low	6.00	6.00	9.77×10^{-4}	9.82×10^{-4}
Combined	**High**	**7.83**	**8.62**	2.16×10^{-5}	7.91×10^{-5}
	Medium	**6.75**	**6.91**	1.26×10^{-4}	3.23×10^{-4}
	Low	**5.75**	**5.91**	9.46×10^{-4}	1.78×10^{-3}

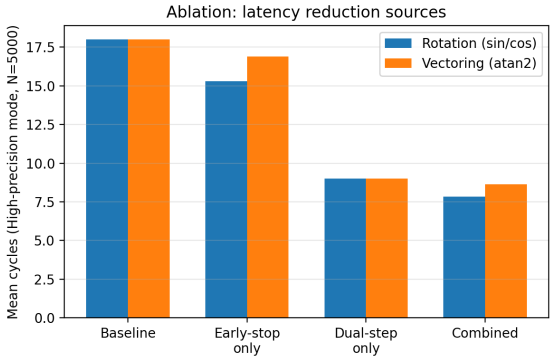


Fig. 4. Ablation study (High-precision mode): mean cycle count for each configuration, both stages.

D. Ablation Results

Table III reports the measured mean cycle count and error statistics for all four configurations across the three precision modes, on the 5,000+5,000 held-out evaluation set described above.

Figure 4 visualizes the High-precision-mode cycle counts across the four configurations for both stages, and Figure 5 shows the resulting accuracy-latency operating points of the combined configuration across all three precision modes.

Averaging the two stages, dual-step scheduling alone reduces mean cycle count from the 18/14/11-cycle fixed baseline to 9/7/6 cycles (a 50.0%, 50.0%, and 45.5% reduction for High/Medium/Low, respectively) with *no change* in numerical error, since it only changes how many micro-iterations are committed per clock rather than which iterations are executed. Early termination alone is comparatively modest in this circular-mode design – 10.6%, 3.9%, and 2.1% for High/Medium/Low – because the chosen nominal depths already sit close to where the residual first crosses the corresponding threshold for most of the sampled inputs; unlike the hyperbolic-mode case, circular-mode convergence is

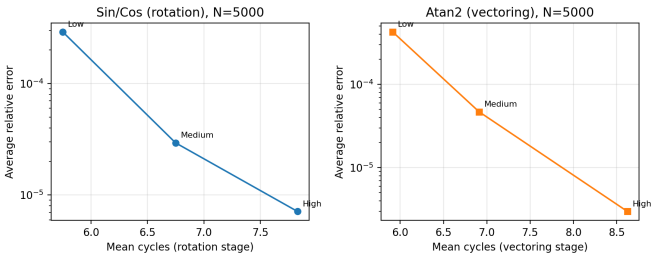


Fig. 5. Accuracy-latency operating points of the combined (DSPC + early-stop) configuration across the three precision modes.

essentially monotonic and does not require the repeated-index correction that would otherwise leave more slack for early exit. Combining both techniques yields the largest cycle-count reduction – 54.3%, 51.2%, and 47.0% for High/Medium/Low – confirming that the two mechanisms are complementary at the *cycle* level: DSPC reduces the per-iteration cycle cost, and early termination reduces the number of iterations actually needed. **This cycle-count picture is, however, incomplete on its own.** Section IV-G carries all four configurations through real technology mapping and static timing analysis, which shows that DSPC's shorter cycle count comes at the cost of a substantially longer critical path – long enough that the combined configuration's real (nanosecond) throughput ends up *lower*, not higher, than the fixed-depth baseline. We defer the full explanation to Section IV-G rather than treat the cycle counts above as a latency claim.

E. Synthesized Logic Cost

Each configuration was first synthesized with Yosys [20], an open-source Verilog synthesis suite, using a technology-independent generic-cell flow (`synth -top cordic_top`), then technology-mapped to the real SkyWater SKY130 high-density standard cell library (`sky130_fd_sc_hd`, typical corner, 25 °C, 1.8 V) via `dfflibmap`/`abc -liberty` – the same synthesis engine used in the open-source OpenLane RTL-to-GDSII flow [21], [22]. Table IV reports the generic-cell counts (technology-independent, useful as a relative complexity measure) alongside the SKY130-mapped cell count and chip area for all four configurations. All configurations map to the same 120 flip-flops; the combinational cell count and area grow with DSPC, reflecting the added second micro-iteration datapath, both-candidate residual comparators, and 2:1 selection between one-step and two-step write-back.

Table V breaks down representative combinational cell types for the Baseline and Combined configurations after SKY130 mapping. The growth is concentrated in two-input Boolean gates (`nand2`, `nor2`, `xor2`, `xnor2`) and the 2:1 multiplexers (`mux2`) used for one-step/two-step write-back selection, consistent with DSPC duplicating the per-iteration adder/subtractor logic rather than adding new sequential state. The static leakage power in Table IV scales with this cell growth (baseline 6.7 nW to combined 10.1 nW, a 50% increase, roughly tracking the 54% area increase); this is a state-averaged leakage figure derived directly from the liberty file's per-cell leakage entries and does not include switching (dynamic) power, which requires an activity-annotated (VCD-based) power analysis we have not carried out here.

TABLE IV
POST-SYNTHESIS CELL COUNT, AREA, AND STATIC LEAKAGE POWER. "GENERIC" IS THE TECHNOLOGY-INDEPENDENT COUNT BEFORE LIBRARY MAPPING; "SKY130" IS AFTER MAPPING TO `SKY130_FD_SC_HD` (TT, 025C, 1.8V). FLIP-FLOP COUNT IS 120 IN ALL FOUR CONFIGURATIONS, BOTH FLOWS. LEAKAGE POWER IS SUMMED FROM THE ACTUAL PER-INSTANCE LIBERTY VALUES IN EACH MAPPED NETLIST (STATE-AVERAGED PER CELL).

Configuration	Generic	SKY130 cells	Area (μm^2)	Leakage (nW)
Baseline	3,218	2,491	17,702.0	6.7
Early-stop only	3,542	2,579	18,686.7	7.0
Dual-step only	5,045	3,797	25,735.9	9.7
Combined	5,565	4,005	27,229.9	10.1

TABLE V
REPRESENTATIVE SKY130-MAPPED CELL-TYPE USAGE, BASELINE VS. COMBINED.

Cell type	Baseline	Combined
`nand2_1`	309	551
`nor2_1`	296	555
`xor2_1`	157	217
`xnor2_1`	157	314
`mux2_1`	18	32
`and2_0`	29	49

F. FPGA-Targeted Resource Utilization

In addition to the SKY130 ASIC flow, we synthesized all four configurations with Yosys's `synth_ice40` flow, which maps to real Lattice iCE40 4-input LUTs, carry cells, and flip-flops rather than technology-independent generic gates. Table VI reports the resulting resource counts. We report this as an FPGA-oriented logic-synthesis resource estimate, not a place-and-routed bitstream result on physical FPGA hardware; no FPGA-vendor Fmax or on-board measurement is claimed.

TABLE VI
FPGA-TARGETED LOGIC SYNTHESIS RESOURCE UTILIZATION (YOSYS `synth_ice40`, LATTICE iCE40 4-INPUT LUT ARCHITECTURE).

Configuration	LUT4	Carry	Flip-flops
Baseline	1,573	628	120
Early-stop only	1,760	694	120
Dual-step only	2,537	887	120
Combined	2,779	1,019	120

G. Real Timing-Closed Throughput (OpenSTA)

The results so far – cycle counts (Table III) and technology-mapped area (Table IV) – do not by themselves establish a real-time latency claim. To close this gap, we built OpenSTA [26] from source against the same `sky130_fd_sc_hd` liberty file (the ABC-internal `stime` pass used in an earlier version of this study failed to parse several clock-related cells in the mapped netlist; OpenSTA's full liberty reader does not have this limitation) and ran static timing analysis directly on each SKY130-mapped netlist from Section IV-F, with a stress-test clock period short enough to report the true critical path via worst negative slack. Table VII reports the resulting critical path delay and maximum clock frequency for all four configurations.

The critical path grows substantially with DSPC – more than doubling from baseline to dual-step-only (11.88 ns to 26.03 ns) – because the second micro-iteration is chained combinationally after the first, and the termination check must additionally evaluate the residual/floor/target condition against *both* candidate states before selecting the write-back mux. Combining the per-mode cycle counts from Table III with the F_{max} values above gives the real operation latency and throughput in Table VIII.

The result is the opposite of what the cycle-count table alone suggests. Despite executing 54%, 51%, and 47% fewer cycles, the Combined configuration's real throughput is *13.4%, 19.0%, and 25.4% lower* than the Baseline's for High, Medium, and Low precision, respectively, because F_{max} drops by 60.5% (84.2 to 33.3 MHz) – proportionally more than the cycle count drops. Dual-step-only shows the same pattern (8–16% lower throughput than Baseline despite 45–50% fewer cycles). Early-stop-only, which does not touch the per-iteration combinational depth, comes closest to preserving Baseline's real performance but still trails it, since its F_{max} also drops (84.2 to 61.3 MHz) due to the added residual-comparator logic on the critical path, by more than its modest cycle savings recover. *Of the two mechanisms proposed in this paper, only residual-based early termination survives contact with real post-synthesis timing as a plausible net improvement candidate at all – and even it does not recover full parity with the fixed-depth baseline in this particular implementation.* We discuss the implications of this finding, and what it would take for DSPC to pay off in practice, in Section V-A.

TABLE VII
OPENSTA-DERIVED CRITICAL PATH DELAY AND MAXIMUM FREQUENCY, SKY130 TT/025C/1.8V CORNER.

Configuration	Critical path (ns)	F_{max} (MHz)
Baseline	11.88	84.2
Early-stop only	16.32	61.3
Dual-step only	26.03	38.4
Combined	30.05	33.3

TABLE VIII
REAL (STA-DERIVED) LATENCY AND THROUGHPUT PER PRECISION MODE. CYCLE COUNTS ARE THE ROTATION/VECTORING AVERAGE FROM TABLE III.

Config.	Mode	Cyc.	Lat. (ns)	Tput. (Mops/s)
Baseline	High	18.00	213.8	4.68
	Medium	14.00	166.3	6.01
	Low	11.00	130.7	7.65
Early-stop only	High	16.09	262.5	3.81
	Medium	13.45	219.5	4.56
	Low	10.77	175.8	5.69
Dual-step only	High	9.00	234.3	4.27
	Medium	7.00	182.2	5.49
	Low	6.00	156.2	6.40
Combined	**High**	8.23	247.2	**4.05**
	Medium	6.83	205.2	**4.87**
	Low	5.83	175.2	**5.71**

H. Word-Length Sensitivity

Because the target depths and thresholds in Table II were tuned at a single fixed-point format (Q4.20, 24-bit nominal operands, 32-bit internal registers), we additionally swept the fractional word length (FRAC = 14–24 bits) against several fixed iteration depths to characterize where accuracy becomes word-length-limited rather than iteration-limited. Figure 6 shows the result for the rotation stage (500 randomized angles).

At 20-bit fractional precision (the format used in Sections III–IV), error at depth 18 or beyond is essentially flat with further word-length increase (max error changes by less than $2\times$ from FRAC=20 to FRAC=24 at depth 18–20), indicating the design has reached the point where *iteration count*, not *register width*, is the binding constraint at the High-precision setting – i.e., the 24-bit operand width chosen for this design is not leaving significant accuracy on the table relative to a wider datapath. At lower depths (12), increasing FRAC beyond 16–18 bits gives essentially no improvement, since the 2^{-12}-scale truncation of the *algorithmic* (iteration-count-limited) error already dominates the smaller $2^{-\mathrm{FRAC}}$-scale rounding error. This confirms that the Low-precision mode's accuracy floor in Table III is set by its target depth choice, not by the 24-bit fixed-point format, and that reducing register width further (e.g. to 16 bits) would not measurably degrade Low-mode accuracy – a possible avenue for additional area reduction at low precision that this design does not currently exploit.

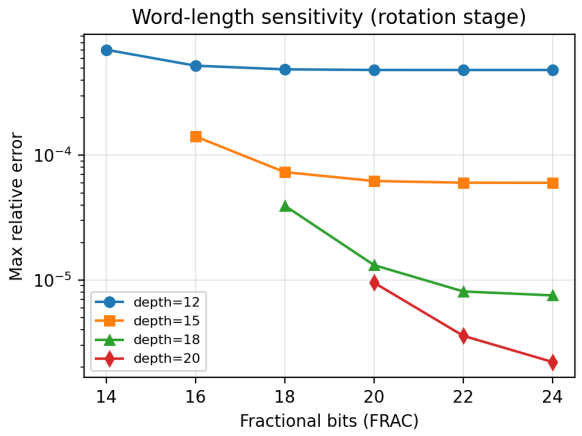


Fig. 6. Word-length sensitivity: maximum relative error vs. fractional bit width, for four fixed iteration depths.

V. Discussion

A. What It Would Take for DSPC to Pay Off

Section IV-G's finding – that DSPC reduces cycles but not real throughput in this implementation – is a statement about *this non-pipelined, combinationally-chained realization of the idea*, not a general refutation of dual-stepping as a technique. Two concrete paths could change the outcome. First, *pipelining* the two micro-iterations across a register stage (rather than chaining them combinationally within one cycle) would let each half-iteration run at closer to the Baseline's per-iteration critical path, recovering most of F_{max} at the cost of a one-cycle pipeline latency and the control complexity of draining the pipeline correctly when early termination fires mid-flight; this is a materially different design from the one evaluated here and is future work. Second, in a system where the clock period is already fixed by a slower, unrelated critical path elsewhere on the same die (a common situation for a CORDIC engine embedded in a larger SoC), *cycle* count – not this engine's own standalone F_{max} – is what determines its contribution to end-to-end latency, and DSPC's cycle reduction would be realized without penalty. Neither condition holds in the standalone, maximum-frequency evaluation used throughout this paper, which is why we report the throughput reversal as the headline timing result rather than the cycle-count reduction.

B. DSPC vs. Higher-Radix CORDIC

Radix-4 and other higher-radix CORDIC variants also reduce cycle count by processing more angle per iteration, but require redundant digit selection and, in most reported designs, extra correction logic to bound the angle-recoding error [10]. DSPC instead keeps the underlying radix-2 recurrence unchanged and simply commits two micro-iterations per clock when the residual has not yet converged, which is why dual-step scheduling in this design costs zero additional numerical error (Table III, identical error columns for Baseline and Dual-step only) – though, as Section IV-G shows, that numerical-cost-free property does not by itself imply a real latency win, since the mechanism still roughly doubles the combinational logic (and hence critical path) of the single-step datapath.

C. Comparison Basis with Lookup-Table and Polynomial Methods

Lookup-table sine/cosine generators, as used in classical direct digital frequency synthesizers [25], [24], trade table depth directly for accuracy and are attractive at low precision but scale poorly in memory as accuracy requirements increase [9]; polynomial and piecewise-polynomial approximants (e.g. Horner-scheme minimax polynomials evaluated with multiply-accumulate hardware [23], hybrid piecewise-polynomial phase-to-amplitude converters [24], or hybrid CORDIC-multiplier synthesizer/mixer designs [18]) avoid the memory cost but reintroduce multiplier hardware, whose area and delay scale with the target word length in a way that shift-add CORDIC avoids. The proposed design retains the multiplier-free shift-add property of CORDIC across all three precision modes, trading only combinational logic (Table IV) for iteration count, which is a more graceful and runtime-reconfigurable trade-off than re-synthesizing a different lookup-table depth, polynomial order, or multiplier width for each target accuracy – though, per Section IV-G, the precision-mode selection itself (which does not enable DSPC's combinational doubling) is the part of this trade-off that actually delivers a real-time benefit; the DSPC scheduling mechanism should be read as a cycle-count/area trade-off whose real-time payoff depends on the deployment context discussed in Section V-A, not as a standing latency improvement.

D. Qualitative Comparison with Representative CORDIC Architectures

A direct quantitative PPA comparison against prior circular-mode CORDIC designs would require matching word length, precision target, target technology, and speed grade to each cited work's own evaluation setup – conditions we cannot establish for designs we have not independently re-implemented on the SKY130/iCE40 flows used here. We therefore do not report side-by-side Fmax/area numbers pulled from other papers' own (differently configured) evaluations, since a table populated that way would create a false impression of a controlled comparison. Table IX instead compares architectural *characteristics* that can be read directly off each work's own description, positioning this design within the design space rather than ranking it by unmatched numbers.

By this comparison, the adaptive-precision mechanism (Section III) is what differentiates this design architecturally – runtime precision/latency selection via a single input, rather than a synthesis-time choice – while DSPC (Section IV-B) is, per Section IV-G's finding, not shown here to be a net advantage over the simpler single-step datapath that most of these prior designs already use. Vendor CORDIC IP such as AMD/Xilinx's LogiCORE CORDIC [27], configurable for rotation, vectoring, and several other functions with either word-serial or fully parallel architectures, remains a natural practical

TABLE IX
QUALITATIVE COMPARISON WITH REPRESENTATIVE CIRCULAR-MODE CORDIC ARCHITECTURES. "ADAPTIVE" MEANS THE DESIGN EXPOSES MORE THAN ONE RUNTIME-SELECTABLE PRECISION/LATENCY OPERATING POINT; "MULT.-FREE" MEANS THE DATAPATH IS SHIFT-ADD ONLY, WITH NO EMBEDDED MULTIPLIER.

Work	Technique	Radix	Mult.-free	Adaptive
This work	Early-term. + DSPC	2	Yes	Yes (3)
Lin & Wu [14]	Mixed-scal.-rotation	2	Yes	No
Strollo [18]	Hybrid CORDIC-mult.	2	No	No
Aggarwal *et al.* [17]	Redesigned scale-free	2	Yes	No
Changela *et al.* [19]	Mixed-radix scale-free	4	Yes	No
Salehi *et al.* [9]	Angle-recoded LL	2	Yes	No
Xilinx Logi-CORE [27]	Word-serial/par.	2	Yes	No*

*Configurable at synthesis/generation time (word length, architecture, iteration count), not selectable at runtime by a live hardware input the way `prec_sel` is here.

reference point: its own architecture description (a coarse quadrant rotation into the first quadrant, matching the folding stage in Section II-B) is methodologically close enough to this design that a direct resource/frequency comparison would be a natural extension once the RTL here is retargeted to a Xilinx device via the vendor toolchain – but we leave that retargeting, and the numeric comparison it would enable, as future work rather than approximate it here.

E. Limitations

Several simplifying choices and scope boundaries should be stated plainly. *Scope.* This paper addresses circular-mode CORDIC (sine, cosine, arctangent) only; the adaptive-precision and DSPC mechanisms are not evaluated here for hyperbolic-mode functions, where the repeated-index correction required for convergence [6] would interact with DSPC's paired-step scheduling in ways this study does not characterize. *Input range.* The quadrant folding in Section II-B assumes the input angle is already wrapped to $[-\pi, \pi)$; a full mod-2π reduction stage for unbounded angle inputs is standard but not included in the area/cycle figures reported here. *Usable operand magnitude.* Section IV-C's boundary testing found that vectoring inputs with magnitude below roughly 2^{-13} in this Q4.20 format produce rapidly growing absolute error (Section IV-C); consequently, applications that vectorize operands routinely near this floor – e.g. small-signal sensor residuals or post-subtraction values in a control loop – should apply input normalization (rescaling to a larger representable magnitude before the CORDIC stage) or a wider internal representation; applications whose inputs stay well above this floor (most angle-tracking and phase-generation uses) are unaffected. *Offline-tuned thresholds.* The three precision modes are runtime-selectable at the hardware interface (via `prec_sel`), but the depth/floor/threshold values themselves (Table II) are fixed constants chosen by an offline sweep against representative test data (Section III-B), not adapted online to the actual input distribution or observed error at run time. *Input distribution.* The ablation study (Section IV-D) and the corner-case testing (Section IV-C) together cover uniform-random and deterministic boundary/axis inputs, but not a specific downstream application's input distribution (e.g. the near-zero-crossing-heavy angle distribution typical of a phase accumulator in a direct digital synthesizer); early-termination savings in particular are sensitive to the input distribution and may differ from Table III for a specific application. *Power.* Table IV's leakage figure is state-averaged static leakage only; switching (dynamic) power, which depends on toggle activity and would be needed for a full energy-per-operation figure, is not measured here. *Register width.* The 120-flip-flop control/state overhead is dominated by the two 32-bit (x, y, z) registers required to avoid overflow (Section IV-C); the word-length sweep in Section IV-H suggests a narrower internal accumulator would not measurably degrade Low-mode accuracy, but this design does not currently exploit that headroom.

VI. CONCLUSION

This paper presented an adaptive-precision, dual-step CORDIC engine unifying circular-mode sine, cosine, and arctangent computation in a single reconfigurable datapath, and evaluated it more rigorously than cycle-count simulation alone: a tuning/evaluation data split, a $10\times$ larger held-out random test set, a deterministic boundary/worst-case test set, real SKY130 and iCE40 synthesis, real per-cell leakage power, and real OpenSTA static timing closure. The boundary testing paid for itself immediately, surfacing a genuine branch-cut bug (a 2π output-angle error at exact-axis vectoring inputs, now fixed) and a precisely quantified fixed-point dynamic-range floor (operand magnitudes below 2^{-13} in the Q4.20 format). On the held-out evaluation set, combining residual-based early termination with dual-step-per-cycle (DSPC) scheduling reduces mean iteration count by 47–54% across three precision modes with error remaining within 1.8×10^{-3} even in the lowest-precision mode. **The central, and more important, finding of this study is that this cycle-count reduction does not survive contact with real post-synthesis timing:** OpenSTA analysis on the SKY130-mapped netlists shows DSPC's chained two-step combinational datapath increases the critical path enough ($2.5\times$ at Combined) that real throughput ends up 13–25% *lower* than a naive fixed-depth, single-step baseline, despite executing far fewer cycles. Residual-based early termination alone – which does not alter the per-iteration combinational depth – comes closest to preserving the baseline's real performance, though it does not fully recover it either in this implementation. We report this negative result for the DSPC mechanism deliberately and in full alongside the positive results, because a hardware paper's cycle-count claims are only as credible as the timing analysis behind them, and we believe this cautionary finding is at least as useful to the CORDIC scheduling literature as the adaptive-precision result itself. The design retains a fully multiplier-free, shift-add datapath throughout, requires no repeated-index correction for

either the rotation or vectoring stage, and all RTL, testbenches, synthesis scripts, and raw results (including the netlists analyzed by OpenSTA) are provided so that the throughput reversal in Table VIII can be independently reproduced or challenged. Future work includes a pipelined DSPC variant that could recover real throughput (Section V-A), extending the adaptive-precision criterion to hyperbolic-mode functions, dynamic (switching-activity-based) power characterization, and a matched-configuration comparison against commercial FPGA CORDIC IP.

REFERENCES

[1] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.

[2] J. S. Walther, "A unified algorithm for elementary functions," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1971, pp. 379–385.

[3] H. M. Ahmed, J.-M. Delosme, and M. Morf, "Highly concurrent computing structures for matrix arithmetic and signal processing," *IEEE Computer*, vol. 15, no. 1, pp. 65–82, Jan. 1982.

[4] R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA)*, Monterey, CA, USA, 1998, pp. 191–200.

[5] "A review of CORDIC algorithm applications for sine and cosine generation in digital signal processing," in *Recent Advances in Signal Processing and Hardware Design*, Springer, 2026.

[6] Y. Luo, Y. Wang, H. Sun, Y. Zha, Z. Wang, and H. Pan, "CORDIC-based architecture for computing Nth-root and its implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 12, pp. 4183–4195, Dec. 2018.

[7] A. Changela, Y. Kumar, M. Woźniak, J. Shafi, and M. F. Ijaz, "Radix-4 CORDIC algorithm based low-latency and hardware efficient VLSI architecture for Nth root and Nth power computations," *Sci. Rep.*, vol. 13, art. no. 20918, 2023.

[8] M. Qin, T. Liu, B. Hou, Y. Gao, Y. Yao, and H. Sun, "A low-latency RDP-CORDIC algorithm for real-time signal processing of edge computing devices in smart grid cyber-physical systems," *Sensors*, vol. 22, art. no. 7489, 2022.

[9] F. Salehi *et al.*, "Novel design for a low-latency CORDIC algorithm for sine-cosine computation and its implementation on FPGA," *Microprocessors and Microsystems*, 2020.

[10] "FPGA implementation of CORDIC algorithms for sine and cosine generator" (hybrid-adaptive, virtually-scaling-free-adaptive CORDIC variants), 2015.

[11] W. Shen, J. Jiang, M. Li, and S. Liu, "Efficient CORDIC-based activation functions for RNN acceleration on FPGAs," *IEEE Trans. Artif. Intell.*, 2024.

[12] N. A. Mohamed and J. R. Cavallaro, "A unified parallel CORDIC-based hardware architecture for LSTM network acceleration," *IEEE Trans. Comput.*, vol. 72, pp. 2752–2766, Oct. 2023.

[13] P. Poczekajlo *et al.*, "Low latency recoding CORDIC algorithm for FPGA implementation," in *Proc. Int. Conf. Comput. Sci. (ICCS)*, 2025.

[14] C.-H. Lin and A.-Y. Wu, "Mixed-scaling-rotation CORDIC (MSR-CORDIC) algorithm and architecture for high-performance vector rotational DSP applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 52, no. 11, pp. 2385–2396, Nov. 2005.

[15] Y. Luo, Y. Wang, Y. Ha, Z. Wang, S. Chen, and H. Pan, "Generalized hyperbolic CORDIC and its logarithmic and exponential computation with arbitrary fixed base," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 9, pp. 2156–2169, Sep. 2019.

[16] Y. Liu, L. Fan, and T. Ma, "A modified CORDIC FPGA implementation for wave generation," *Circuits Syst. Signal Process.*, vol. 33, pp. 321–329, 2014.

[17] A. Aggarwal *et al.*, "Redesigned-scale-free CORDIC algorithm based FPGA implementation of window functions to minimize area and latency," *Int. J. Reconfigurable Comput.*, vol. 2012, art. no. 185784, 2012.

[18] A. G. M. Strollo, "Digital synthesizer/mixer with hybrid CORDIC-multiplier architecture: error analysis and optimization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 56, no. 2, pp. 364–373, Feb. 2009.

[19] A. Changela *et al.*, "Mixed-radix, virtually scaling-free CORDIC algorithm based rotator for DSP applications," *J. Syst. Archit.*, 2021.

[20] C. Wolf, "Yosys – a free Verilog synthesis suite," in *Proc. 21st Austrian Workshop Microelectron. (Austrochip)*, 2013.

[21] M. Shalan and T. Edwards, "Building OpenLane: A 130nm OpenROAD-based tapeout-proven flow," in *Proc. 39th Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–6.

[22] T. Ajayi *et al.*, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Proc. GOMACTECH*, 2019.

[23] F.-G. Stoica, A. Calinescu, and M. Enachescu, "ROM-less co(sine) synthesizer," *Electronics*, vol. 15, no. 5, art. no. 1093, 2026.

[24] Q. K. Omran, M. T. Islam, N. Misran, and M. R. I. Faruque, "A ROM-less direct digital frequency synthesizer based on hybrid polynomial approximation," *Int. J. Antennas Propag.*, vol. 2014, art. no. 812576, 2014.

[25] S. H. Ibrahim, S. H. Md. Ali, and M. S. Islam, "Hardware implementation of 32-bit high-speed direct digital frequency synthesizer," *The Scientific World Journal*, vol. 2014, art. no. 131568, 2014.

[26] J. Sartori, R. Cofell, and others (The OpenROAD Project), "OpenSTA: static timing analyzer," *GitHub repository*, https://github.com/The-OpenROAD-Project/OpenSTA, accessed 2026.

[27] AMD/Xilinx, "CORDIC v6.0 LogiCORE IP Product Guide," PG105, Aug. 2021.